

Kalman-Based Temporal Domain Adaptation in 3D Synthetic Drift Environments

This project evaluates whether temporal domain drift can be modeled as a predictable trajectory using Kalman filtering over neural decision boundaries.

The benchmark uses rotating 3D Gaussian blobs with chirp-style temporal drift and compares several forecasting/adaptation strategies: - Static pooled model - Linear extrapolation in anchor-output space - LSTM prediction in weight space - Kalman filtering with uniform weighting - Kalman filtering with forgetting-factor weighting - Last-domain upper-bound model

Overview

The core idea is: 1. Train a neural classifier independently on each temporal domain 2. Evaluate each classifier on a fixed 3D anchor grid 3. Treat the resulting probability vectors as temporal observations 4. Fit a Kalman filter to the sequence of observations 5. Predict the next domain's decision boundary 6. Adapt the classifier toward the predicted future boundary

The experiment tests whether temporal structure in evolving domains can improve future-domain performance.

Synthetic Drift Environment

The dataset consists of: - Two 3D Gaussian blobs - Rotating decision geometry - Chirp-style temporal acceleration - Domain-specific stochastic drift noise

Drift evolves according to:

$$\theta(t) = 2\pi \left(f_0 t + \frac{(f_1 - f_0)t^2}{2(T - 1)} \right) + \epsilon$$

where:

- f_0 and f_1 define chirp acceleration.
 - ϵ is Gaussian domain noise.
-

Methods Compared

1. Static pooled model

Single model trained on all observed domains.

2. Linear extrapolation

Fits linear trends independently for each anchor probability.

3. LSTM weight prediction

Predicts future neural network weights directly in parameter space.

4. Standard Kalman filtering

Kalman filtering on anchor-output trajectories.

5. Weighted Kalman filtering

Uses forgetting-factor weighting so recent domains contribute more strongly.

6. Last-domain model

Upper-bound baseline trained directly on the immediately preceding domain.

Anchor Point Representation

Each classifier is evaluated on a fixed 3D anchor grid:

- Grid size: $6 \times 6 \times 6$
- Total anchors: 216

For each anchor: - model outputs $P(y = 1 \mid x)$

The resulting vector becomes the observation for Kalman filtering.

Dependencies

Install:

```
pip install numpy matplotlib torch pykalman
```

Running

Run the experiment:

```
python kalman_3d_chirp.py
```

Outputs: - Cross-validation accuracy summaries - Per-domain performance curves - Visualization figures - Kalman adaptation comparisons

Main Files

File	Purpose
<code>kalman_3d_chirp.py</code>	Main synthetic temporal adaptation experiment

Key Research Question

Can temporal drift trajectories be predicted using decision-boundary dynamics instead of retraining directly on future labeled data?

Notes

- The weighted Kalman filter generally performs better under noisy temporal drift.
- Linear extrapolation is a strong baseline in smooth synthetic settings.
- The last-domain model is an upper bound and is not directly comparable to forecasting methods.